Protocolo:

/*
Tecnológico Nacional de México
Instituto Tecnológico de León
Ingeniería en Sistemas Computacionales
Estructuras de datos
Lunes y miércoles 7:00 a 8:45
Viernes 7:00 a 7:50
Alumno: Villagómez Magaña Máximo Javier
Tarea 1
Fecha: 19/4/2026
*/

Diagrama UML – Paquetes

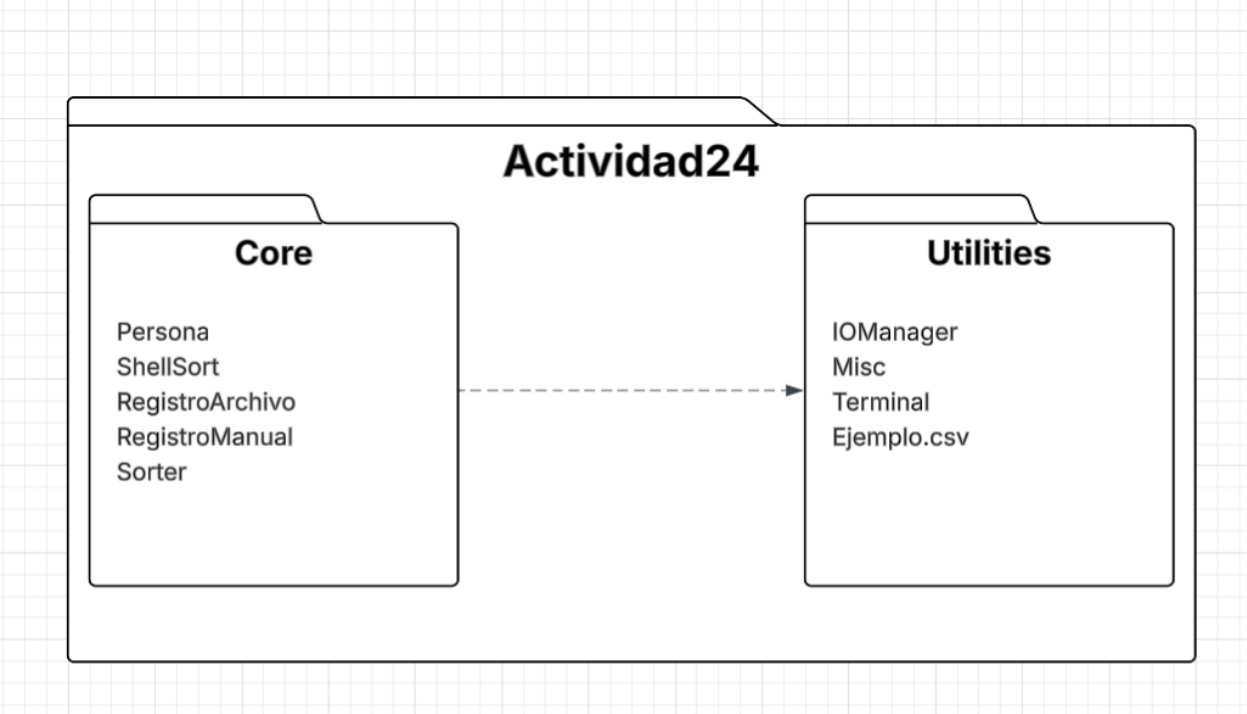
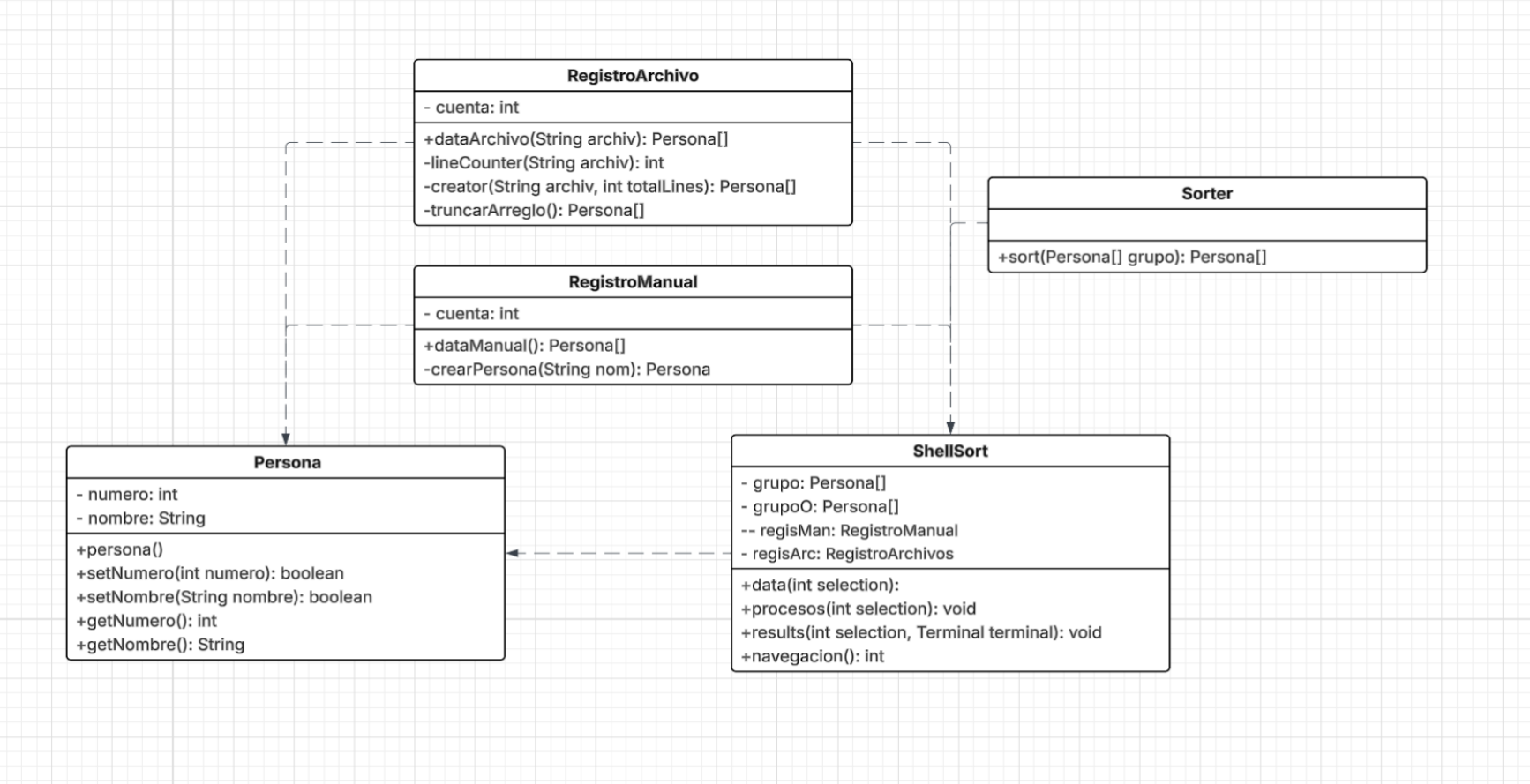


Diagrama UML – Paquetes



Algoritmo – ShellSort (Principal)

```
1. METODO goal(terminal):
  msg = "Código que busca poner a prueba las prestaciones del sorteo burbuja"
  terminal.imprimir(msg)

2. METODO data(selection):
  SEGUN (selection) EMPEZAR
    CASO 1: grupo = regisMan.dataManual()
    CASO 2:
      Escribe("ENTRANDO AL CASO 2")
      path = IOManager.fileSearcher()
      grupo = regisArc.dataArchivo(path)
  TERMINAR

3. METODO procesos(selection):
  grupoO = Sorter.sort(grupo)

4. METODO results(selection, terminal):
  SEGUN (selection) EMPEZAR
    CASO 3:
      DESDE i = 0 HASTA (grupoO.length - 1) EMPEZAR
        msg = grupoO[i].getNumero() + "|" + grupoO[i].getNombre()
        terminal.imprimir(msg)
      TERMINAR
    CASO 4:
      IOManager.fileWritter("resultado_ordenado.csv", grupoO)
  TERMINAR

5. METODO navegacion():
  msg = "Seleccione una de las siguientes opciones: "
  msg = msg + "1.- Registro manual | 2.- Registro archivo "
  msg = msg + "3.- Mostrar en terminal | 4.- Pasar a archivo | 5.- Salir"
```

```
  REGRESA IOManager.menuCreator(msg, 5)

6. METODO main(args):
  prueba = NUEVA ShellSort()
  terminal = NUEVA Terminal("Terminal de impresión")
  prueba.goal(terminal)

  REPETIR EMPEZAR
    option = prueba.navegacion()
    SI (option != 5) EMPEZAR
      prueba.data(option)
      prueba.procesos(option)
      prueba.results(option, terminal)
    TERMINAR
  MIENTRAS (option != 5)
```

Algoritmo – Sorter (Extensión de principal)

```
1. METODO sort(grupo):
  n = grupo.length

  // Algoritmo ShellSort: Bucle externo para el intervalo (gap)
  MIENTRAS (gap > 0) EMPEZAR
    gap = n / 2
    // Bucle medio para recorrer los elementos
    DESDE i = gap HASTA (n - 1) EMPEZAR
      temp = grupo[i]
      j = i

      // Bucle interno para insertar el elemento en su lugar
      MIENTRAS (j >= gap Y grupo[j - gap].getNumero() > temp.getNumero()) EMPEZAR
        grupo[j] = grupo[j - gap]
```

```
        j = j - gap
      TERMINAR
      grupo[j] = temp
    TERMINAR
    gap = gap / 2 // Reducción del intervalo
  TERMINAR
  REGRESA grupo
```

Algoritmo – RegistroManual (Extensión de principal)

```
1. METODO dataArchivo(archiv):
  INTENTAR EMPEZAR
    totalLines = lineCounter(archiv)
    cuenta = 0
    grupo = creator(archiv, totalLines)
    REGRESA truncarArreglo(grupo, cuenta)
  TERMINAR
  ATRAPANDO (Exception e) EMPEZAR
    Escribe("Error al procesar el archivo" + e.getMessage())
    REGRESA null
  TERMINAR

2. METODO lineCounter(archiv):
  totalLines = 0
  contador = NUEVO Scanner(NUEVO File(archiv))
  MIENTRAS (contador.hasNextLine()) EMPEZAR
    contador.nextLine()
    totalLines = totalLines + 1
  TERMINAR
  contador.close()
  REGRESA totalLines
```

3. METODO creator(archiv, totalLines):

```
grupo = NUEVO Persona[totalLines]
lector = NUEVO Scanner(NUEVO File(archiv))

MIENTRAS (lector.hasNextLine()) EMPEZAR
  lineaActual = lector.nextLine()
  info = lineaActual.split(",")

  SI (info.length >= 2) EMPEZAR
    nPersona = NUEVO Persona()
    d1 = nPersona.setNumero(Integer.parseInt(info[0]))
    d2 = nPersona.setNombre(info[1])

    // Si un solo dato es invalido, se toma como basura
    SI (d1 Y d2) EMPEZAR
      grupo[cuenta] = nPersona
      cuenta = cuenta + 1
    TERMINAR
  TERMINAR
TERMINAR
lector.close()
REGRESA grupo
```

4. METODO truncarArreglo(original, cantidadReal):

```
limpio = NUEVO Persona[cantidadReal]
System.arraycopy(original, 0, limpio, 0, cantidadReal)
REGRESA limpio
```

Algoritmo – RegistroArchivo (Extensión de principal)

```
1. METODO dataManual():

  cuenta = 0

  grupo = NUEVO Persona[25]


  nom = Misc.entryString("Proporcione el nom de la persona " + (cuenta + 1) + " o escriba 'fin' para terminar")

  MIENTRAS (nom != "fin" Y cuenta < grupo.length) EMPEZAR

    grupo[cuenta] = crearPersona(nom)

    cuenta = cuenta + 1

    Escribe("Persona registrada con exito")

    SI (cuenta < grupo.length) EMPEZAR

      nom = Misc.entryString("Proporcione el nom de la persona " + (cuenta + 1) + " o escriba 'fin' para
terminar")

    TERMINAR

  TERMINAR

  REGRESA grupo


2. METODO crearPersona(nom):

  p = NUEVA Persona()

  Misc.numChecker("De # de control: ", p)

  REGRESA p
```

Código – Java - ShellSort

```
package Entregables.actividades21_24.actividad24.Core;
import Entregables.actividades21_24.Utilities.IOManager;
import Entregables.actividades21_24.Utilities.Terminal;

public class ShellSort {
    private Persona[] grupo;
    private Persona[] grupoO;
    private RegistroManual regisMan = new RegistroManual();
    private RegistroArchivo regisArc = new RegistroArchivo();

    public void goal(Terminal terminal) {
        String msg = "Codigo que busca poner a ";
```

```
    msg += "prueba las prestaciones del sorteo burbuja";
    terminal.imprimir(msg);
}

public void data(int selection) {
    switch(selection) {
        case 1 -> grupo = regisMan.dataManual();
        case 2 -> {
            System.out.println("ENTRANDO AL CASO 2");
            String path = IOManager.fileSearcher();
            grupo = regisArc.dataArchivo(path);
        }
    }
}

public void procesos(int selection) {
    grupoO = Sorter.sort(grupo);
}

public void results(int selection, Terminal terminal) {
    String msg;
    switch(selection) {
        case 3 -> {
            for (int i = 0;i< grupoO.length;i++) {
                msg = grupoO[i].getNumero() + "\\|" + grupoO[i].getNombre();
                terminal.imprimir(msg);
            }
        }
        case 4 -> IOManager.fileWritter("resultado_ordenado.csv", grupoO);
    }
}

public int navegacion() {
    StringBuilder msg = new StringBuilder();
    msg.append("Seleccione una de las siguientes opciones\n ");
    msg.append("1.- Registro de datos manual\n");
    msg.append("2.- Registro de datos por archivo\n");
    msg.append("3.- Mostrar resultados ordenados en terminal\n");
    msg.append("4.- Pasar resultados ordenados a archivo\n");
    msg.append("5.- Salir");
    return IOManager.menuCreator(msg, 5);
}

public static void main(String[] args) {
    ShellSort prueba = new ShellSort();
    Terminal terminal = new Terminal("Terminal de impresion");
```

```
        int option;

        prueba.goal(terminal);
        do {
            option = prueba.navegacion();
            if (option != 5) {
                prueba.data(option);
                prueba.procesos(option);
                prueba.results(option, terminal);
            }
        } while (option != 5);
    }
}
```

Código – Java – Sorter

```
package Entregables.actividades21_24.actividad24.Core;

public class Sorter {
    public static Persona[] sort(Persona[] grupo) {
        int n = grupo.length;

        for (int gap = n / 2; gap > 0; gap /= 2) {

            for (int i = gap; i < n; i++) {
                Persona temp = grupo[i];
                int j;

                for (j = i; j >= gap && grupo[j - gap].getNumero() > temp.getNumero(); j -= gap) {
                    grupo[j] = grupo[j - gap];
                }

                grupo[j] = temp;
            }
        }
        return grupo;
    }
}
```

Código – Java – RegistroManual

```
package Entregables.actividades21_24.actividad24.Core;
import Entregables.actividades21_24.Utilities.Misc;
import javax.swing*;

public class RegistroManual {
    private int cuenta = 0;
```

```
    public Persona[] dataManual() {
        cuenta = 0;
        Persona[] grupo = new Persona[25];
        String nom;

        nom = Misc.entryString("Proporcione el nom de la persona " +(cuenta+1)+
                        "o escriba \"fin\" para terminar");

        while (!nom.equalsIgnoreCase("fin")  && this.cuenta < grupo.length) {
            grupo[cuenta] = crearPersona(nom);
            this.cuenta++;
            JOptionPane.showMessageDialog(null,
                "Persona registrada con exito");

                if (cuenta < grupo.length) {
                nom = Misc.entryString("Proporcione el nom de la persona " +(cuenta+1)+
                    "o escriba \"fin\" para terminar");
                }
            }
    return grupo;
    }

    private Persona crearPersona(String nom) {
        Persona p = new Persona();
        Misc.numChecker("De # de control: ",p);
        return p;
    }
}
```

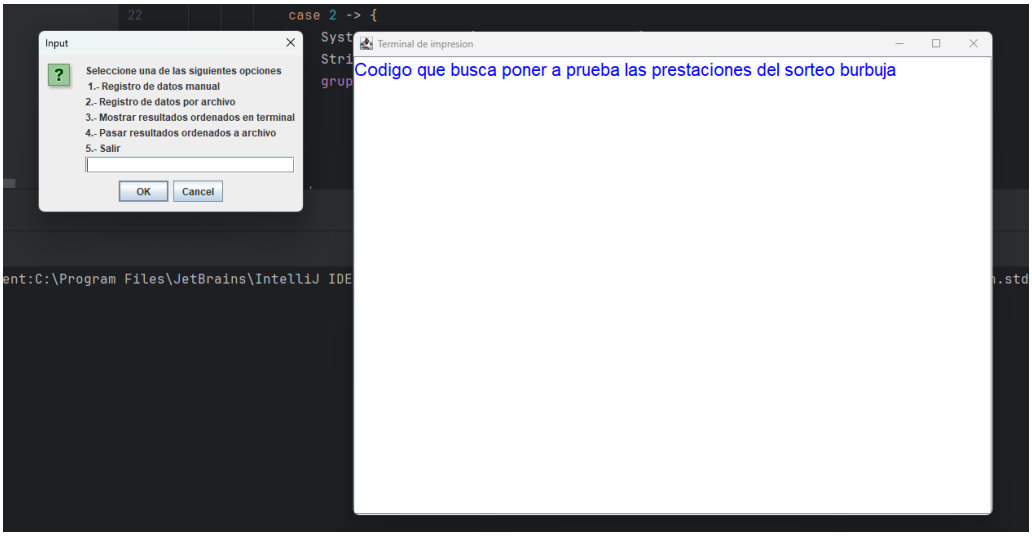
Código – Java - RegistroArchivo

```
package Entregables.actividades21_24.actividad24.Core;

import javax.swing.*;
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class RegistroArchivo {
    private int cuenta = 0;

    public Persona[] dataArchivo(String archiv) {
        int totalLines;

        try {
            totalLines = lineCounter(archiv);
            this.cuenta = 0;
```

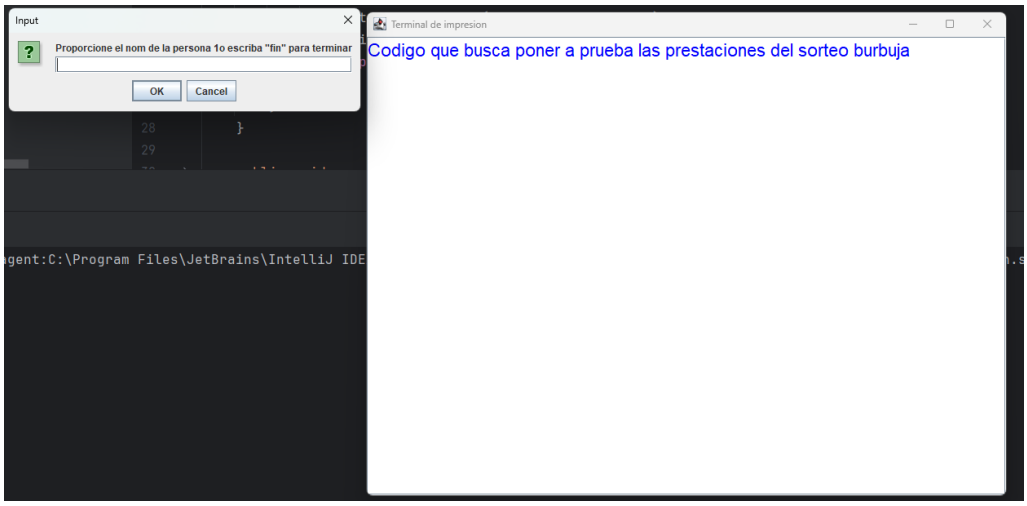
```
        Persona[] grupo = creator(archiv, totalLines);

        return truncarArreglo(grupo, this.cuenta);
    } catch (Exception e) {
        JOptionPane.showMessageDialog(null,
                "Error al procesar el archivo"+ e.getMessage());
        return null;
    }
}

private int lineCounter(String archiv) throws FileNotFoundException {
    int totalLines = 0;

    Scanner contador = new Scanner(new File(archiv));
    while (contador.hasNextLine()) {
        contador.nextLine();
        totalLines++;
    }
    contador.close();
    return totalLines;
}

private Persona[] creator(String archiv, int totalLines) throws FileNotFoundException {
    Persona[] grupo = new Persona[totalLines];
    Scanner lector = new Scanner(new File(archiv));

    while (lector.hasNextLine()) {
        String lineaActual = lector.nextLine();
        String[] info = lineaActual.split(",");

        if (info.length >= 2) {
            Persona nPersona = new Persona();
            boolean d1 = nPersona.setNumero(Integer.parseInt(info[0]));
            boolean d2 = nPersona.setNombre(info[1]);

            //Si un solo dato es invalido, se toma como basura
            if (d1 && d2) {
                grupo[this.cuenta] = nPersona;
                this.cuenta++;
            }
        }
    }
    lector.close();
    return grupo;
}
```

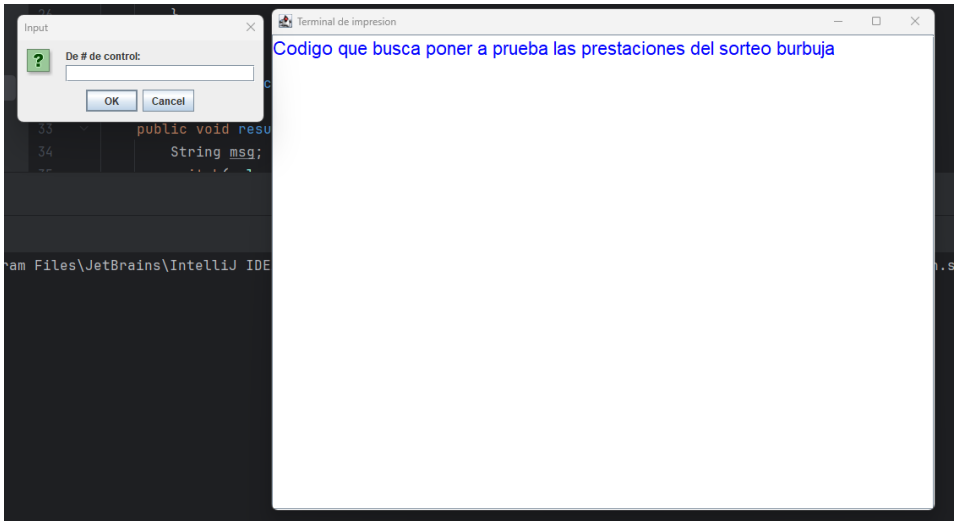
```
    private Persona[] truncarArreglo(Persona[] original, int cantidadReal) {
        Persona[] limpio = new Persona[cantidadReal];
        System.arraycopy(original, 0, limpio, 0, cantidadReal);
        return limpio;
    }
}
```

Resultado de la corrida

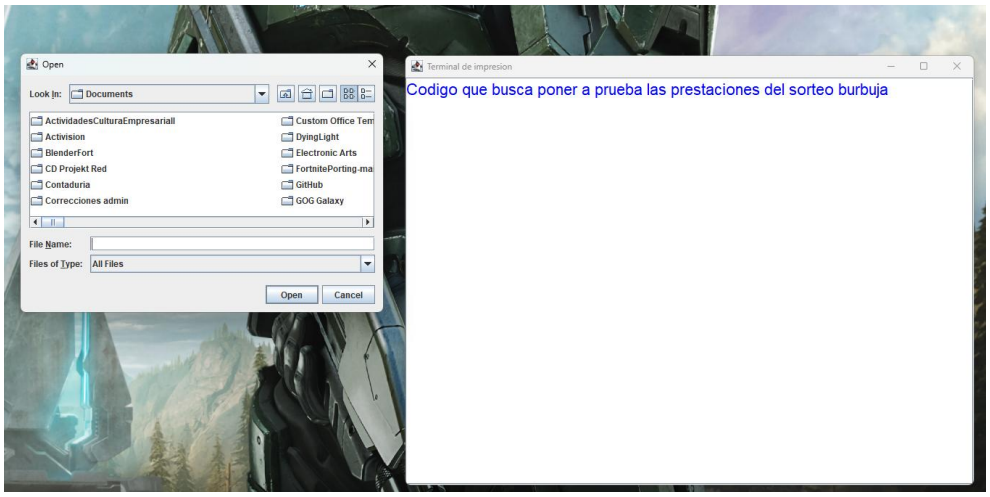


1.-

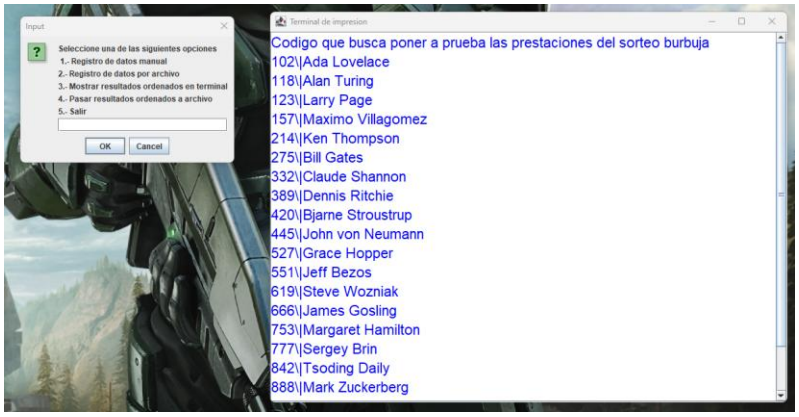



Input
De # de control:
OK
Cancel
public void resu
String msg;
am Files\JetBrains\IntelliJ IDE
Terminal de impresion
Codigo que busca poner a prueba las prestaciones del sorteo burbuja

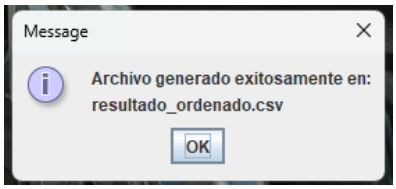
2.-


Open
Look In:
Documents
ActividadesCulturaEmpresariall
Activision
BlenderFort
CD Projekt Red
Contaduria
Correcciones admin
Custom Office Tem
DyingLight
Electronic Arts
FortnitePorting-ma
GitHub
GOG Galaxy
File Name:
Files of Type:
All Files
Open
Cancel
Terminal de impresion
Codigo que busca poner a prueba las prestaciones del sorteo burbuja

3.-


Input
Seleccione una de las siguientes opciones
1.- Registro de datos manual
2.- Registro de datos por archivo
3.- Mostrar resultados ordenados en terminal
4.- Pasar resultados ordenados a archivo
5.- Salir
OK
Cancel
Terminal de impresion
Codigo que busca poner a prueba las prestaciones del sorteo burbuja
102\|Ada Lovelace
118\|Alan Turing
123\|Larry Page
157\|Maximo Villagomez
214\|Ken Thompson
275\|Bill Gates
332\|Claude Shannon
389\|Dennis Ritchie
420\|Bjarne Stroustrup
445\|John von Neumann
527\|Grace Hopper
551\|Jeff Bezos
619\|Steve Wozniak
666\|James Gosling
753\|Margaret Hamilton
777\|Sergey Brin
842\|Tsoding Daily
888\|Mark Zuckerberg

4.-



Archivo no ordenado

ejemplo.csv | misc.java | ShellSort.java | resultado_ordenado.csv | Sorter.java | RegistroMa

```
842,Tsoding Daily
157,Maximo Villagomez
931,Linus Torvalds
420,Bjarne Stroustrup
666,James Gosling
102,Ada Lovelace
527,Grace Hopper
389,Dennis Ritchie
214,Ken Thompson
753,Margaret Hamilton
118,Alan Turing
901,Guido van Rossum
445,John von Neumann
332,Claude Shannon
619,Steve Wozniak
275,Bill Gates
888,Mark Zuckerberg
551,Jeff Bezos
123,Larry Page
777,Sergey Brin
```

Archivo Nuevo

ejemplo.csv | misc.java | ShellSort.java | resultado_ordenado.csv

```
102,Ada Lovelace
118,Alan Turing
123,Larry Page
157,Maximo Villagomez
214,Ken Thompson
275,Bill Gates
332,Claude Shannon
389,Dennis Ritchie
420,Bjarne Stroustrup
445,John von Neumann
527,Grace Hopper
551,Jeff Bezos
619,Steve Wozniak
666,James Gosling
753,Margaret Hamilton
777,Sergey Brin
842,Tsoding Daily
888,Mark Zuckerberg
901,Guido van Rossum
931,Linus Torvalds
```